Algoritmo e Laboratório de Programação II

Aula 4 – Novos Tipos de Dados com struct, typedef, union e enum - parte 2 (Nível Avançado)

Roteiro

- As funções atoi() e atof()
- A função exit()
- Usando typedef para tipo de funções
- Tipos de dados enumerados: enum
- Usando typedef com enum
- Uniões
- Usando typedef com union
- Uniões de estruturas
- O operador sizeof
- O operador de endereços (&) com union
- Uniões anônimas

As funções atoi() e atof()

- Decidimos apresentar duas novas funções da biblioteca C para que você as conheça.
- A função atoi(), com protótipo no arquivo stdlib.h, recebe o endereço de uma string como argumento e o converte para um valor inteiro.
- Caso a string não contenha um número válido, ela retornará zero.
- Espaços em branco iniciais são ignorados.

As funções atoi() e atof()

- A função atof(), com protótipo no arquivo stdlib.h, recebe o endereço de uma string como argumento e o converte a um valor double.
- Caso a string não contenha um número válido, ela retornará zero.
- Espaços em branco iniciais são ignorados.

Roteiro

- As funções atoi() e atof()
- A função exit()
- Usando typedef para tipo de funções
- Tipos de dados enumerados: enum
- Usando typedef com enum
- Uniões
- Usando typedef com union
- Uniões de estruturas
- O operador sizeof
- O operador de endereços (&) com union
- Uniões anônimas

A função exit()

- A função exit() da biblioteca-padrão, com protótipo no arquivo stdlib.h, permite o término imediato do programa e passa o controle ao sistema operacional.
  - A função exit() finaliza o programa().
- O seu argumento é um número inteiro que é retornado ao sistema operacional e pode ser acessado por meio do subcomando ERRORLEVEL de um programa .BAT.

A função exit()

- O arquivo .BAT pode conter uma linha como:
- IF ERRORLEVEL 1 GOTO ERR1
- O que provoca uma mudança de operação, dependendo do valor de retorno indicado pelo término do programa.
- O zero costuma ser usado para indicar o término normal do programa.

Roteiro

- As funções atoi() e atof()
- A função exit()
- Usando typedef para tipo de funções
- Tipos de dados enumerados: enum
- Usando typedef com enum
- Uniões
- Usando typedef com union
- Uniões de estruturas
- O operador sizeof
- O operador de endereços (&) com union
- Uniões anônimas

Usando typedef para tipo de funções

- O exemplo seguinte cria o tipo FuncType() para uma função que não retorna nada e recebe um float e um int como argumentos, nessa ordem:

```
typedef void FuncType(float, int);
```

- Após essa instrução, a próxima declaração é válida: `FuncType MinhaFunc;`
- E é equivalente à seguinte instrução:

```
void MinhaFunc(float, int);
```

Roteiro

- As funções atoi() e atof()
- A função exit()
- Usando typedef para tipo de funções
- Tipos de dados enumerados: enum
- Usando typedef com enum
- Uniões
- Usando typedef com union
- Uniões de estruturas
- O operador sizeof
- O operador de endereços (&) com union
- Uniões anônimas

Tipos de dados enumerados: enum

- Os tipos definidos com enum, chamados tipos enumerados, consistem num conjunto de constantes inteiras, em que cada uma delas é representada por nome.
- Tipos enumerados são usados quando conhecemos o conjunto de valores que uma variável pode assumir.
- A variável desse tipo é sempre int e, para cada um dos valores do conjunto, atribuímos um nome significativo.

Tipos de dados enumerados: enum

- A palavra enum enumera a lista de nomes automaticamente, dando-lhes números em sequência (0, 1, 2, etc.).
- A vantagem é que utilizamos esses nomes no lugar de números, o que torna o programa mais claro.
- Vamos escrever um fragmento de programa que define o tipo enumerado e cria uma variável desse tipo para guardar o mês.

Tipos de dados enumerados: enum

Prog6.c

```c
#include <stdio.h>

enum Mes {Jan = 1, Fev, Mar, Abr, Mai, Jun, Jul, Ago, Set, Out,
          Nov, Dez
         };

int main()
{
    enum Mes m1, m2, m3; /*Cria variáveis do tipo enum Mes */

    m1 = Abr; /*Atribui valores */
    m2 = Jun; /*Atribui valores */

    m3 = m2 - m1; /*Operações aritméticas permitidas */
    if (m1 < m2) /*Comparações permitidas */
    {
        printf("Primeiro Mês é menor que o Segundo Mês");
    }
    else
    {
        printf("Segundo Mês é menor que o Primeiro Mês");
    }
    return 0;
}
```

Tipos de dados enumerados: enum

- A palavra enum define um conjunto de nomes (cada um deles com um valor inteiro), enumera esses nomes a partir do zero (default) ou, como em nosso programa, a partir do primeiro valor fornecido.
- Se fornecermos um valor a alguns nomes e não a outros, o compilador atribuirá o próximo valor inteiro aos que não tiverem valor.

Tipos de dados enumerados: enum

- Uma variável de um tipo enumerado pode ter qualquer valor listado na definição.
- Não podemos atribuir valores não listados.

```
m1 = sabado; /* ERRADO. É ilegal */
```

- Tipos enumerados são tratados internamente como inteiros; portanto, qualquer operação possível com inteiros é permitida com eles.

Roteiro

- As funções atoi() e atof()
- A função exit()
- Usando typedef para tipo de funções
- Tipos de dados enumerados: enum
- Usando typedef com enum
- Uniões
- Usando typedef com union
- Uniões de estruturas
- O operador sizeof
- O operador de endereços (&) com union
- Uniões anônimas

Usando typedef com enum

Prog7.c

```c
#include <stdio.h>

typedef enum Mes {Jan = 1, Fev, Mar, Abr, Mai, Jun, Jul, Ago, Set, Out,
                  Nov, Dez
                 };

int main()
{
    Mes m1, m2, m3; /*Agora o nome do tipo é Mes */

    m1 = Abr; /*Atribui valores */
    m2 = Jun; /*Atribui valores */

    m3 = m2 - m1; /*Operações aritméticas permitidas */
    if (m1 < m2) /*Comparações permitidas */
    {
        printf("Primeiro Mês é menor que o Segundo Mês");
    }
    else
    {
        printf("Segundo Mês é menor que o Primeiro Mês");
    }
    return 0;
}
```

Usando typedef com enum

- Outros exemplos:

```
typedef enum {Feminino, Masculino} sexo;
typedef enum {OFF, ON} chave;
```

- Se o modo como o compilador atribui valores não se adapta ao que você quer, poderá especificar os seus próprios valores:

```
typedef enum {Norte = 50, Sul = 80, Leste = 92, Oeste = 100} Direcao;
```

Roteiro

- As funções atoi() e atof()
- A função exit()
- Usando typedef para tipo de funções
- Tipos de dados enumerados: enum
- Usando typedef com enum
- Uniões
- Usando typedef com union
- Uniões de estruturas
- O operador sizeof
- O operador de endereços (&) com union
- Uniões anônimas

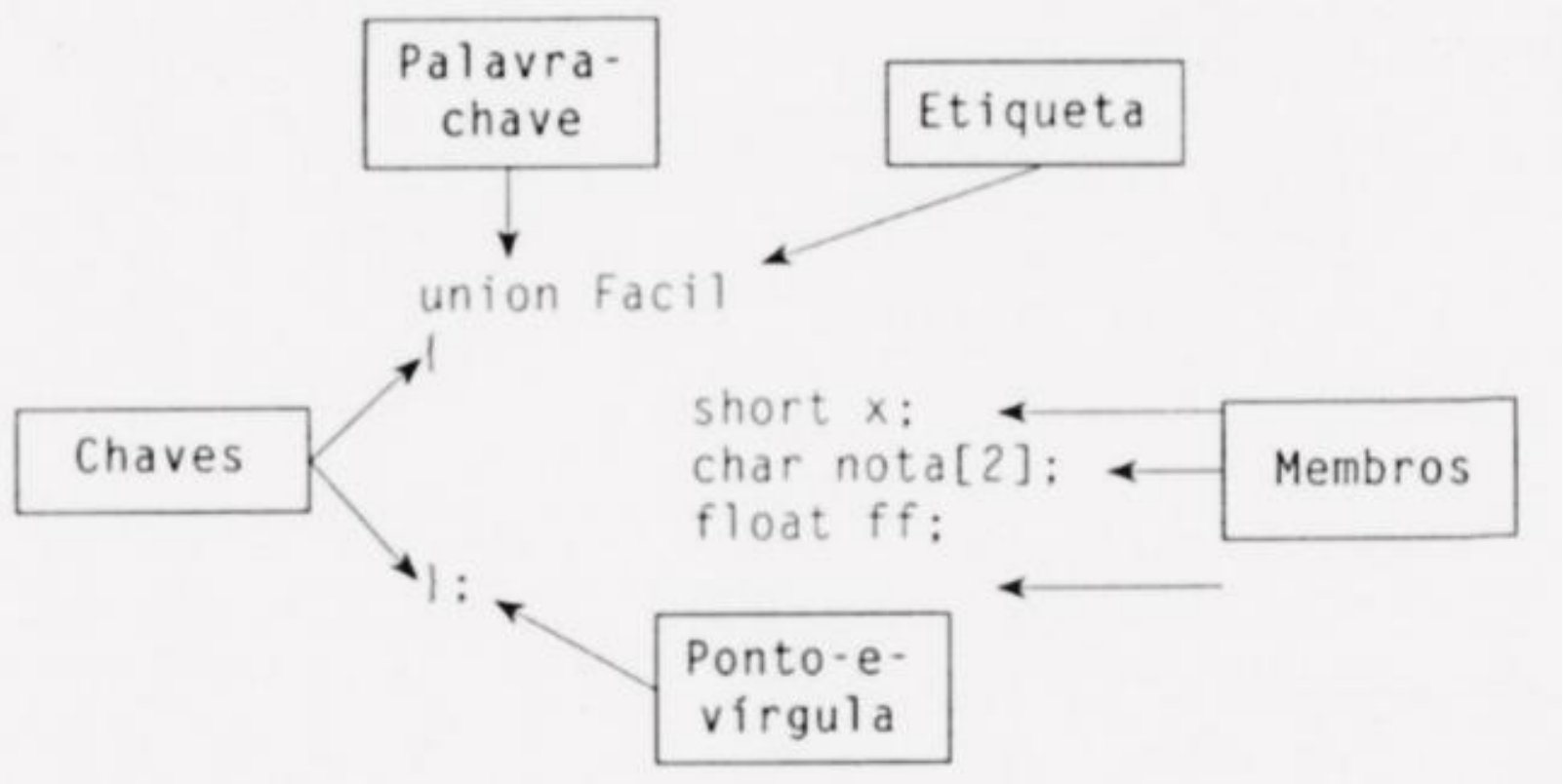
Uniões

- A sintaxe de definição de novos tipos de dados por meio da palavra Union é idêntica à da palavra struct.
- Entretanto, uma union utiliza um mesmo espaço de memória a ser compartilhado com diferentes membros, enquanto uma struct aloca um espaço diferente de memória para cada membro.

Uniões

- Em outras palavras, uma union é o meio pelo qual um pedaço de memória é tratado ora como uma variável de certo tipo, ora como outra variável de outro tipo.
- Uniões podem ser usadas para poupar memória.
- Quando você declara uma variável de um tipo union, automaticamente é alocado um espaço de memória suficiente para conter o seu ,maior membro.

Uniões



- Veja um exemplo:

Uniões

Prog8.c

```c
/* Prog8.c */
/* Mostra o uso de uniões */
#include <stdio.h>
#include <stdlib.h>
#include <string.h> /* Para strcpy() */

union Numero
{
    char str[30];
    int i;
    float f;
};

union Numero x; /* Cria variável */
```

Uniões

```
16  enum DataType
17  {
18      StrType, IntType, FloatType
19  } Tipo; /* Cria variável */
20
21  void StrTipo(char n[])
22  {
23      Tipo=StrType;
24      strcpy(x.str, n);
25  }
26  void FloatTipo(float n)
27  {
28      Tipo=FloatType;
29      x.f=n;
30  }
31  void IntTipo(int n)
32  {
33      Tipo=IntType;
34      x.i=n;
35  }
```

Uniões

```
37  void Print()
38  {
39      switch(Tipo)
40      {
41      case StrType:
42          printf("Str = %s\n", x.str);
43          break;
44      case IntType:
45          printf("Int = %d\n", x.i);
46          break;
47      case FloatType:
48          printf("Float = %.2f\n", x.f);
49          break;
50      }
51  }
```

Uniões

```
53  int main()
54  {
55      float y = 34.56;
56      int z = 345;
57
58      StrTipo("VVB-Treinamento de Pessoal");
59      Print();
60      IntTipo(z);
61      Print();
62
63      FloatTipo(y);
64      Print();
65
66      system("PAUSE");
67      return 0;
68  }
```

Uniões

- A seguir, a saída do programa:

```
Str = VVB - Treinamento de Pessoal
Int = 345
Float = 35
```

- Uniões são tipos de dados que armazenam um único membro por vez.
- Os membros de uma union representam os diversos tipos de dados que ela pode conter.

Roteiro

- As funções atoi() e atof()
- A função exit()
- Usando typedef para tipo de funções
- Tipos de dados enumerados: enum
- Usando typedef com enum
- Uniões
- Usando typedef com union
- Uniões de estruturas
- O operador sizeof
- O operador de endereços (&) com union
- Uniões anônimas

Usando typedef com union

- A sintaxe de union é a mesma de struct.
- Assim, podemos usar typedef.

```
typedef union
{
    char str[30];
    int i;
    float f;
} Numero;

Numero x; /* Cria variável */
```

Usando typedef com union

- Combinando declarações:
  - Podemos combinar a definição da union com a declaração da variável.

```
union Numero
{
    char str[30];
    int i;
    float f;
} x; /* Cria variável */
```

  - Podemos declarar quantas variáveis quisermos:

```
union Numero
{
    char str[30];
    int i;
    float f;
} x, y, z; /* Cria variável */
```

Usando typedef com union

- Combinando declarações:
  - A etiqueta pode ser suprimida:

```
union
{
    char str[30];
    int i;
    float f;
} x; /* Cria variável */
```

- Inicializando union:
  - Estruturas e matrizes static ou extern podem ser inicializadas quando são definidas;
  - Desse modo, seria razoável pensar que o mesmo é permitido para uniões.

Usando typedef com union

- Inicializando union:
  - Uma união tem um único membro ativo em qualquer tempo e seria conveniente pensar na seguinte inicialização:

```
static union
{
    int idade;
    float salario;
} uInit = {123.456};
```

Usando typedef com union

- Inicializando union:
  - O compilador deveria ser capaz de entender que estamos inicializando a variável uInit por meio do membro float salario.
  - Entretanto, muitos compiladores não permitem a inicialização de uniões e, quando permitem, ela é aplicada apenas ao primeiro membro declarado.

Usando typedef com union

- Inicializando union:
  - Nesse caso, se você quer inicializar uInit.salario, a declaração de união deve ser modificada:

```
static union
{
    float salario; /* Deve ser definido primeiro */
    int idade;
} uInit = {123.456};
```

Roteiro

- As funções atoi() e atof()
- A função exit()
- Usando typedef para tipo de funções
- Tipos de dados enumerados: enum
- Usando typedef com enum
- Uniões
- Usando typedef com union
- Uniões de estruturas
- O operador sizeof
- O operador de endereços (&) com union
- Uniões anônimas

Uniões de estruturas

- Uma união pode ter membros de qualquer tipo, até mesmo estruturas ou outras uniões.
- Uma união também pode ser membro de uma estrutura.
- Veja um exemplo:

Uniões de estruturas

Prog9.c

```c
/* Prog9.c */
/* Mostra união de estruturas */
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    struct doisint
    {
        int n1;
        int n2;
    };

    union intflo
    {
        struct doisint x;
        float   f;
    } unex;
```

Uniões de estruturas

```
20      printf("sizeof(union intflo) = %d\n", sizeof(union intflo));
21      unex.x.n1 = 734;
22      unex.x.n2 = 333;
23
24      printf("unex.x.n1       = %d\n" , unex.x.n1);
25      printf("unex.x.n2       = %d\n" , unex.x.n2);
26
27      unex.f = 345.22;
28
29      printf("unex.f          = %.2f\n", unex.f );
30
31      system("PAUSE");
32      return 0;
33  }
```

Uniões de estruturas

- A saída será:

```
sizeof(union intflo) = 8
unex.x.n1      = 734
unex.x.n2      = 333
unex.f         = 345.22
```

- Observe que o operador ponto foi usado duas vezes para acessar os membros da estrutura interna x.
- Vamos mostrar o uso de uma estrutura de campo bits como membro de uma union.
- Eis um exemplo que permite convertes um caractere em seu correspondente binário:

Uniões de estruturas

Prog10.c

```c
/* Prog10.c */
/* Mostra união com estrutura */
/* Imprime um caractere em binário */
#include <stdio.h>
#include <stdlib.h>

typedef struct
{
    /* cada membro com um bit de tamanho */
    unsigned char bit0: 1; /* bit 0 */
    unsigned char bit1: 1; /* bit 1 */
    unsigned char bit2: 1; /* bit 2 */
    unsigned char bit3: 1; /* bit 3 */
    unsigned char bit4: 1; /* bit 4 */
    unsigned char bit5: 1; /* bit 5 */
    unsigned char bit6: 1; /* bit 6 */
    unsigned char bit7: 1; /* bit 7 */
} Bits;
```

Uniões de estruturas

```
20  typedef union
21  {
22      unsigned char ch;
23      Bits x;
24  } Binario;
25
26  int main(void)
27  {
28      Binario B;
29      printf("Digite um caractere: ");
30      B.ch  = getchar();
31      printf("O caractere digitado é: %c\n", B.ch);
32      printf("O valor em binário correspondente é: ");
33
34      printf("%d", B.x.bit7);
35      printf("%d", B.x.bit6);
36      printf("%d", B.x.bit5);
37      printf("%d", B.x.bit4);
38      printf("%d", B.x.bit3);
39      printf("%d", B.x.bit2);
40      printf("%d", B.x.bit1);
41      printf("%d", B.x.bit0);
42      printf("\n");
```

Uniões de estruturas

```
44      system("pause");
45      return 0;
46  }
```

- A seguir a saída do programa:

```
Digite um caractere: A
O caractere digitado é: A
O valor em binário correspondente é: 01000001
```

Roteiro

- As funções atoi() e atof()
- A função exit()
- Usando typedef para tipo de funções
- Tipos de dados enumerados: enum
- Usando typedef com enum
- Uniões
- Usando typedef com union
- Uniões de estruturas
- O operador sizeof
- O operador de endereços (&) com union
- Uniões anônimas

O operador sizeof

- Uma variável de um tipo union tem o tamanho do maior membro.
- Para verificar isso, usaremos um novo operador C: sizeof.
- O operador sizeof opera sobre o nome de um tipo de dado ou sobre o nome de uma variável e resulta o seu tamanho em bytes.
- Veja um exemplo:

O operador sizeof

Prog11.c

```c
/* Prog11.c */
/* Mostra o uso do operador sizeof() */
#include <stdio.h>
#include <stdlib.h>

union Numero
{
    char str[32];
    int i;
    float f;
} x; /* Cria variável */

int main(void)
{
    printf("%d\n", sizeof(union Numero));
    printf("%d\n", sizeof(x));

    system("PAUSE");
    return 0;
}
```

O operador sizeof

- A expressão

```
sizeof(union Numero)
```

- É equivalente a

```
sizeof(x)
```

- E resulta o tamanho do tipo union Numero.

Roteiro

- As funções atoi() e atof()
- A função exit()
- Usando typedef para tipo de funções
- Tipos de dados enumerados: enum
- Usando typedef com enum
- Uniões
- Usando typedef com union
- Uniões de estruturas
- O operador sizeof
- O operador de endereços (&) com union
- Uniões anônimas

O operador de endereços (&) com union

- O endereço de memória de cada membro de uma união é o mesmo, visto que todos compartilham o mesmo espaço.
- Podemos verificar isso utilizando o operador de endereços.

O operador de endereços (&) com union

Prog12.c

```c
/* Prog12.c */
/* Mostra o uso do operador de endereço */
#include <stdio.h>
#include <stdlib.h>

typedef union
{
    int i;
    float f;
    double d;
} Num;

int main(void)
{
    Num exemplo;

    printf("\n\n");
    printf("Endereço de i  = %08X\n" , &exemplo.i );
    printf("Endereço de f  = %08X\n" , &exemplo.f );
    printf("Endereço de d  = %08X\n" , &exemplo.d );
    printf("Endereço de x  = %08X\n" , &exemplo   );

    system("PAUSE");
    return 0;
}
```

O operador de endereços (&) com union

- No programa, usamos o operador de endereços com cada um dos membros da união e também como o próprio nome da variável união.
- A expressão

```
&exemplo
```

- Resulta o endereço do primeiro byte ocupado pela variável exemplo.

Roteiro

- As funções atoi() e atof()
- A função exit()
- Usando typedef para tipo de funções
- Tipos de dados enumerados: enum
- Usando typedef com enum
- Uniões
- Usando typedef com union
- Uniões de estruturas
- O operador sizeof
- O operador de endereços (&) com union
- Uniões anônimas

Uniões anônimas

- Uniões anônimas são uniões definidas sem a especificação do nome do tipo.
- Eis um exemplo:

Prog13.c

```c
/*Prog13.c */
/* Mostra o uso de uniões anônimas */
#include <stdio.h>
#include <stdlib.h>
#include <string.h> /* Para strcpy() */

enum DataType { StrType, IntType, FloatType};

typedef struct
{
    enum DataType Tipo;
    union
    {
        char str_valor[30];
        int int_valor;
        float float_valor;
    };
} UniaoLiteral;
```

Uniões anônimas

```
38  void Print(UniaoLiteral x)
39  {
40      switch(x.Tipo)
41      {
42      case StrType:
43          printf("Str = %s\n",x.str_valor);
44          break;
45      case IntType:
46          printf("Int = %d\n",x.int_valor);
47          break;
48      case FloatType:
49          printf("Float = %.2f\n",x.float_valor);
50          break;
51      }
52  }
```

Uniões anônimas

```
20  void Print(UniaoLiteral);
21
22  int main()
23  {
24      UniaoLiteral A;
25      A.Tipo=StrType;
26      strcpy(A.str_valor, "VVB-Treinamento de Pessoal");
27      Print(A);
28      A.Tipo=FloatType;
29      A.float_valor=34.56;
30      Print(A);
31      A.Tipo=IntType;
32      A.int_valor=345;
33      Print(A);
34
35      system("PAUSE");
36      return 0;
37  }
```

Uniões anônimas

- Em C, uniões anônimas só podem ser definidas como membros de estruturas.
- Elas são acessadas diretamente, por meio do nome da variável struct, como se fossem membros da estrutura.
- Quando criamos uma união anônima, devemos tomar cuidado de indicar a cada tempo qual é o membro da union corrente e lembrar que existirá somente um membro de cada vez.

Referências

- MIZRAHI, Victorine V. **Treinamento em Linguagem C**, São Paulo: Pearson Prentice Hall, 2008;